

Project 1: University Management System using OOP

Description

Create a University Management System that allows administrators to manage students, courses, and faculty members. The system should allow students to enroll in courses, view their course schedules, and check their grades. Faculty members should be able to view their course assignments and student rosters.

Concepts Covered

1. **Class in Python:** Define classes for Student, Course, Faculty, and University to represent the different entities in the system.
2. **Objects in Python:** Create objects for each class to represent individual students, courses, faculty members, and the university.
3. **Polymorphism in Python:** Implement polymorphic methods for enroll and view_schedule that can be applied to different types of students (e.g., undergraduate, graduate).
4. **Encapsulation in Python:** Use encapsulation to hide the implementation details of the Student and Faculty classes and provide a public interface for administrators to interact with them.
5. **Inheritance in Python:** Use inheritance to create a hierarchy of student types (e.g., UndergraduateStudent inherits from Student) and to create a Department class that inherits from University.
6. **Data Abstraction in Python:** Use data abstraction to represent complex data structures (e.g., course schedules, student rosters) in a simplified way, making it easier to work with the data.

Project 2: Employee Management System

Description

Create an Employee Management System that demonstrates the concepts of inheritance, polymorphism, encapsulation, static methods, class methods, and operator overloading in Python OOP.

Concepts Covered

1. **Inheritance:** The Manager and Developer classes inherit from the Employee class.
2. **Polymorphism:** The details() method is overridden in the Manager and Developer classes to provide specific behavior.
3. **Encapsulation:** The __name attribute is private in the Employee class and can only be accessed through the display() and details() methods.
4. **Static Methods:** The get_employee_count() method is a static method that returns the total number of employees. It can be called without creating an instance of the class.
5. **Class Methods:** The get_employee_count_classmethod() method is a class method that returns the total number of employees. It can be called without creating an instance of the class, and it has access to the class variables.
6. **Operator Overloading:** The __str__, __eq__, __lt__, and __gt__ methods are overloaded to provide custom behavior for the print(), ==, <, and > operators, respectively.